区块链技术及应用技能赛

赛前培训资料

区块 1　区块 2　区块 3　区块 4　区块 5

参赛选手内部培训资料

2026 年安徽省区块链技术及应用技能赛

内 部 资 料

严禁外传 • 禁止复制 • 仅供参赛选手使用

内部资料

安徽省电子学会 • 安徽省计算机学会 • 巢湖学院 • 中科晶格
2026 年 5 月

保 密 声 明

本资料为内部培训资料，仅供参赛选手使用。
严禁以任何形式外传、复制、拍照或上传至互联网。
违者将取消参赛资格，并追究相关责任。

Contents

1 区块链零基础入门 4
- 1.1 计算机基础知识 . . . 4
  - 1.1.1 进制与编码 . . . 4
  - 1.1.2 网络基础 . . . 4
  - 1.1.3 数据库基础 . . . 4
  - 1.1.4 JSON 格式 . . . 5
- 1.2 职业道德与法律法规 . . . 5
  - 1.2.1 职业道德基本知识 . . . 5
  - 1.2.2 相关法律法规 . . . 5
- 1.3 通证与资产应用（补充） . . . 6
  - 1.3.1 什么是通证（Token）? . . . 6
  - 1.3.2 通证的分类 . . . 6

2 区块链核心概念 6
- 2.1 什么是区块链？ . . . 6
- 2.2 区块链 vs 传统数据库 . . . 7
- 2.3 区块链的类型 . . . 7

3 密码学基础 8
- 3.1 Hash 函数（哈希函数） . . . 8
  - 3.1.1 什么是 Hash？ . . . 8
  - 3.1.2 区块链中的 Hash 应用 . . . 8
  - 3.1.3 常用 Hash 算法 . . . 9
  - 3.1.4 Python 计算 Hash . . . 9
- 3.2 非对称加密（公钥加密） . . . 9
  - 3.2.1 核心概念 . . . 9
  - 3.2.2 椭圆曲线加密（ECC） . . . 10
- 3.3 数字签名 . . . 10
  - 3.3.1 签名过程 . . . 10
  - 3.3.2 区块链中的签名应用 . . . 11
- 3.4 Merkle Tree（默克尔树） . . . 11
  - 3.4.1 结构和原理 . . . 11
  - 3.4.2 Merkle Tree 的优势 . . . 11

4 区块链底层数据结构 12
- 4.1 区块结构 . . . 12
  - 4.1.1 区块头（Block Header） . . . 12
  - 4.1.2 区块体（Block Body） . . . 12
- 4.2 交易结构（UTXO 模型） . . . 13
  - 4.2.1 什么是 UTXO？ . . . 13
  - 4.2.2 交易过程 . . . 13

内部资料

5 共识算法 13
5.1 拜占庭将军问题 . . . 14
5.2 PoW —工作量证明（Proof of Work） . . . 14
5.2.1 核心思想 . . . 14
5.2.2 PoW 的特点 . . . 14
5.3 PoS —权益证明（Proof of Stake） . . . 15
5.3.1 核心思想 . . . 15
5.3.2 PoS 的特点 . . . 15
5.4 DPoS —委托权益证明 . . . 16
5.4.1 核心思想 . . . 16
5.4.2 DPoS 的特点 . . . 16
5.5 PBFT —实用拜占庭容错 . . . 16
5.5.1 核心思想 . . . 16
5.5.2 三阶段协议 . . . 16
5.5.3 PBFT 的特点 . . . 17
5.6 Raft —非拜占庭容错共识 . . . 17
5.6.1 核心思想 . . . 17
5.6.2 三种角色 . . . 18
5.6.3 Raft vs PBFT . . . 18

6 智能合约 18
6.1 什么是智能合约？ . . . 18
6.1.1 生活化类比 . . . 18
6.2 以太坊和 EVM . . . 19
6.2.1 以太坊的诞生 . . . 19
6.2.2 EVM（以太坊虚拟机） . . . 19
6.2.3 Gas 机制 . . . 19
6.3 Solidity 合约示例 . . . 20

7 区块链的类型 20
7.1 公链（Public Blockchain） . . . 20
7.2 联盟链（Consortium Blockchain） . . . 21
7.3 私链（Private Blockchain） . . . 21

8 区块链应用领域 21
8.1 数字货币与金融应用 . . . 21
8.1.1 比特币（Bitcoin） . . . 21
8.1.2 稳定币（Stablecoin） . . . 22
8.1.3 央行数字货币（CBDC） . . . 22
8.2 资产数字化与供应链金融 . . . 22
8.2.1 资产数字化（Asset Tokenization） . . . 22
8.2.2 供应链金融 . . . 23
8.3 政务与存证 . . . 24
8.3.1 电子政务 . . . 24
8.3.2 司法存证 . . . 24
8.4 数字身份 . . . 25

内部资料

9 中国区块链发展与国密算法 25
9.1 国密算法概述 . . . 25
9.2 SM2 —非对称加密算法 . . . 25
9.2.1 什么是 SM2? . . . 25
9.2.2 SM2 的核心概念 . . . 25
9.2.3 SM2 的两大功能 . . . 26
9.3 SM3 —Hash 算法 . . . 26
9.3.1 什么是 SM3? . . . 27
9.3.2 SM3 vs SHA-256 . . . 27
9.3.3 SM3 的计算过程（简化） . . . 27
9.4 SM4 —对称加密算法 . . . 28
9.4.1 什么是 SM4? . . . 28
9.4.2 SM4 vs AES 对比 . . . 29
9.4.3 SM4 的加密过程 . . . 29
9.4.4 SM4 的工作模式 . . . 29
9.4.5 SM4 的实际应用 . . . 30
9.5 国密算法总结 . . . 31

10 数据结构基础 31
10.1 链表（Linked List） . . . 31
10.2 树（Tree） . . . 32
10.2.1 二叉树 . . . 32
10.2.2 Merkle Tree . . . 32
10.2.3 MPT（Merkle Patricia Tree） . . . 32

11 链上身份（DID） 32
11.1 什么是 DID? . . . 32
11.1.1 DID 的格式 . . . 32
11.1.2 DID 的生成 . . . 33
11.1.3 DID 文档（DID Document） . . . 33
11.2 可验证凭证（VC） . . . 33

12 实操技能备考要点 34
12.1 实操技能要点 . . . 34
12.2 Python 编程要点 . . . 34
12.2.1 Hash 计算 . . . 34
12.2.2 Merkle Root 计算 . . . 35
12.2.3 简单区块链实现 . . . 35
12.3 区块链浏览器实操 . . . 37
12.3.1 比特币浏览器使用 . . . 37
12.3.2 以太坊浏览器使用 . . . 37
12.4 数字签名实操 . . . 38
12.4.1 ECDSA 签名与验证（Python 实现） . . . 38
12.4.2 SM2 签名与验证（国密算法） . . . 39
12.5 备考建议 . . . 40

13 附录 A：核心术语速查表 41

内部资料

14 附录 B：知识体系思维导图 **42**

15 附录 C：推荐学习资源 **43**

16 附录 D：实操环境搭建指南 **43**
16.1 Python 环境准备 43
16.2 区块链浏览器使用 43

17 附录 E：模拟试题 **43**
17.1 理论知识题 43
17.2 实操技能题 44
17.3 答案速查 45

内部资料

1 区块链零基础入门

1.1 计算机基础知识

本节梳理最核心的计算机基础概念，可能以选择题或判断题形式出现。

1.1.1 进制与编码

- 二进制（**Bin**）：计算机内部只用 0 和 1 两种状态，比特币的 Hash 运算是二进制层面的计算
- 十六进制（**Hex**）：二进制太长，用十六进制表示更简洁。SHA-256 的输出就是 64 个十六进制字符
- 字符编码：UTF-8 是互联网上最通用的字符编码，中文在区块链系统中以 UTF-8 存储
- **Base58** 编码：比特币地址使用 Base58Check 编码，去掉了容易混淆的字符（0、O、l、I），更 human-readable

记住：SHA-256 输出 64 个十六进制字符 = 256 位二进制 = 32 字节

1.1.2 网络基础

- **IP** 地址：互联网上的”门牌号”，区块链节点通过 IP 互相通信
- **DNS**：域名系统，把人类可读的网址（如 google.com）转成 IP 地址
- **HTTP/HTTPS**：网页传输协议，区块链浏览器使用 HTTPS 保证安全
- **P2P** 网络：点对点网络，区块链的基础架构。没有中心服务器，所有节点平等

1.1.3 数据库基础

- 关系型数据库：MySQL、Oracle，用表格存储数据，有行和列
- 键值对数据库：Redis、LevelDB，用 Key-Value 存储，区块链常用
- 分布式数据库：数据分散存储在多台机器上，区块链是特殊的分布式数据库
- **ACID** 特性：原子性、一致性、隔离性、持久性。传统数据库追求 ACID，区块链追求最终一致性

1.1.4 JSON 格式

区块链中大量数据以 JSON 格式传输和存储。

```
{
  "name": "Alice",
  "age": 25,
  "isStudent": false,
  "courses": ["Math", "CS", "Blockchain"]
}
```

重点提示：JSON 格式看起来像这样——“k̈ey: v̈alue, n̈umber: 123”。以太坊的智能合约调用和大多数区块链浏览器都返回 JSON 数据。

1.2 职业道德与法律法规

1.2.1 职业道德基本知识

- 诚实守信：区块链行业强调透明和可追溯，从业者必须诚实记录交易和数据
- 保护用户隐私：虽然区块链数据公开，但用户的真实身份需要保护，不能随意泄露
- 持续学习：区块链技术更新快，从业者需要不断学习新技术和新标准
- 安全意识：私钥管理、智能合约安全是区块链的核心，任何疏忽都可能导致巨大损失

1.2.2 相关法律法规

- 《中华人民共和国密码法》（2020 年 1 月 1 日实施）
  - 规范密码应用和管理，促进密码事业发展
  - 密码分为核心密码、普通密码和商用密码
  - 区块链中使用的 SM2/SM3/SM4 属于商用密码
- 《区块链信息服务管理规定》（2019 年 2 月 15 日实施）
  - 要求区块链信息服务提供者进行备案
  - 禁止利用区块链传播违法信息
  - 要求对使用者进行真实身份信息认证
- 《数据安全法》（2021 年 9 月 1 日实施）
  - 规范数据处理活动，保障数据安全
  - 重要数据需要分类分级保护
- 《个人信息保护法》（2021 年 11 月 1 日实施）
  - 规范个人信息处理活动
  - 区块链上存储个人信息需要符合法律规定

内部资料

重点提示：法律部分需要知道有哪些规定存在，以及规定的核心目的（如密码法是为了规范商业密码使用）。不需要背诵条文。

1.3 通证与资产应用（补充）

通证（Token）是区块链的独特概念。

1.3.1 什么是通证（Token）?

简单说：通证就是记录在区块链上的”权益凭证”。

通证的三个特征：

1. 可流通：可以在区块链上自由转让（如转让 NFT 所有权）
2. 可验证：任何人都可以在链上验证其真伪（查区块链即可）
3. 可编程：可以通过智能合约设定规则（自动分红、限量发行等）

类比：门票就是一种通证——有编号（可验证）、可转让（给他人）、有时效（可编程）。

1.3.2 通证的分类

- 同质化通证（**FT**，**Fungible Token**）：每个 Token 都一样，可以互换。类比人民币——每张 100 元价值相等。代表：比特币（BTC）、以太坊（ETH）
- 非同质化通证（**NFT**，**Non-Fungible Token**）：每个 Token 唯一，不可互换。类比房产证——每个房产证对应一套唯一的房子。代表：数字艺术品、游戏道具
- 证券化通证（**STO**）：代表真实资产的所有权（如房地产、股票），受证券法规管

重点：BTC/ETH = 同质化通证（FT），每个都一样；NFT = 非同质化通证，每个都唯一。

2 区块链核心概念

2.1 什么是区块链？

区块链（Blockchain）本质上是一个分布式数据库，但它和普通数据库有一个本质区别：数据一旦写入，就几乎不可能被修改或删除。

用更通俗的话说：区块链就是一个大家共同维护的、无法作弊的电子账本。

从记账场景理解

传统模式：你把钱存银行，银行在它的服务器上记账。你只能相信银行不会篡改你的记录。

区块链模式：所有参与者都保存一份完整的账本副本。任何人修改了自己的账本，其他人的账本都会拒绝匹配。就像全班同学都抄了一份课程表，有人偷偷改了自己的，老师一比对就知道谁在说谎。

区块链的核心特点：

1. 去中心化：没有单一的控制者，所有节点平等
2. 不可篡改：数据一旦写入，修改成本极高（需要控制超过 51% 的节点）
3. 可追溯：每一笔交易都有完整的历史记录，可以追踪资金流向
4. 透明性：所有交易对参与者公开（虽然身份可能是匿名的）

2.2 区块链 vs 传统数据库

LightBG 特性	传统数据库	区块链
数据修改	管理员可以随时修改	几乎不可能修改
控制权	中心化（公司/机构控制）	去中心化（共识机制决定）
信任基础	相信中心机构	相信密码学和共识机制
查询速度	快（毫秒级）	慢（秒级甚至分钟级）
存储成本	低	高（每个节点都存一份）
适合场景	高频交易、大数据	价值转移、存证溯源

区块链不是万能的！它牺牲了效率换取了信任。如果你的场景不需要”去信任”，传统数据库可能更合适。

2.3 区块链的类型

- 公链（**Public Blockchain**）：任何人都可以参与，完全去中心化
  - 代表：比特币、以太坊
  - 特点：匿名、开放、抗审查
  - 共识：PoW、PoS
- 联盟链（**Consortium Blockchain**）：多个机构共同维护，部分去中心化
  - 代表：Hyperledger Fabric、R3 Corda、FISCO BCOS
  - 特点：需要许可、效率较高、适合企业
  - 共识：PBFT、Raft

- 私链（**Private Blockchain**）：单一机构控制，中心化
  - 代表：企业内部系统
  - 特点：完全可控、效率最高、去中心化程度低
  - 用途：内部审计、数据共享

中国主要发展联盟链，因为：1）符合监管要求；2）性能更高；3）适合企业级应用。比特币、以太坊是公链的代表。

3 密码学基础

密码学是区块链的基石。区块链中几乎所有的安全机制都依赖于密码学。本章内容占比很高，务必理解原理。

3.1 Hash 函数（哈希函数）

3.1.1 什么是 Hash？

Hash 函数是一个单向函数，可以把任意长度的输入转换成固定长度的输出。

```
输入："Hello World"
SHA-256输出：
    a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e

输入："Hello World!"（只多了一个感叹号）
SHA-256输出：7
    f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069
```

Hash 函数的核心特性：

1. 单向性：知道输出，无法反推出输入（除非暴力破解）
2. 确定性：同样的输入，永远得到同样的输出
3. 雪崩效应：输入哪怕改变 1bit，输出完全不同
4. 抗碰撞：很难找到两个不同的输入，产生相同的输出

3.1.2 区块链中的 Hash 应用

1. 区块链接：每个区块存储前一个区块的 Hash，形成链式结构
2. 交易指纹：每笔交易计算 Hash 作为唯一标识

3. 数据完整性：任何数据篡改都会导致 Hash 变化，立即被发现
4. 工作量证明（**PoW**）：矿工通过计算 Hash 来竞争记账权

3.1.3 常用 Hash 算法

- **SHA-256**：比特币使用的算法，输出 256 位（64 个十六进制字符）
- **SM3**：中国国密标准，输出 256 位，与 SHA-256 安全强度相当
- **Keccak-256**：以太坊使用的算法（SHA-3 标准）
- **RIPEMD-160**：比特币地址生成中使用的算法，输出 160 位

重点：SHA-256 输出是 64 个十六进制字符（每字符 =4bit，$64 \times 4 = 256$bit）。SM3 是中国国密标准，与 SHA-256 安全强度相当。区块链中优先学习 SHA-256 和 SM3。

3.1.4 Python 计算 Hash

```python
import hashlib

# SHA-256计算
data = b"Hello Blockchain"
hash_result = hashlib.sha256(data).hexdigest()
print(f"SHA-256: {hash_result}")

# 验证雪崩效应
data1 = b"blockchain"
data2 = b"blockchaiN"  # 只改一个大写字母
h1 = hashlib.sha256(data1).hexdigest()
h2 = hashlib.sha256(data2).hexdigest()
print(f"原始: {h1}")
print(f"修改: {h2}")
print(f"不同字符数: {sum(c1 != c2 for c1, c2 in zip(h1, h2))}")
```

3.2 非对称加密（公钥加密）

3.2.1 核心概念

非对称加密使用一对密钥：

- 公钥（**Public Key**）：可以公开分享，用于加密和验证签名
- 私钥（**Private Key**）：必须保密，用于解密和创建签名

非对称加密的两个用途：

1. 加密通信：A 用 B 的公钥加密，只有 B 的私钥能解密
2. 数字签名：A 用私钥签名，任何人可以用 A 的公钥验证

3.2.2 椭圆曲线加密（ECC）

区块链主要使用椭圆曲线加密（ECC），而不是 RSA。

- 优势：相同安全强度下，ECC 的密钥更短、计算更快
  - 256 位 ECC ≈ 3072 位 RSA 的安全强度
  - 比特币、以太坊使用 secp256k1 曲线
  - 中国国密 SM2 使用 sm2p256v1 曲线
- 比特币的密钥生成：
  1. 生成 256 位随机数作为私钥
  2. 使用 ECC 点乘法从私钥生成公钥
  3. 对公钥进行 Hash 运算生成地址

重点：256 位 ECC ≈ 3072 位 RSA 的安全强度。区块链（比特币、以太坊）使用 ECC（secp256k1 曲线）。中国使用 SM2（sm2p256v1 曲线）。公钥可以分享，私钥必须保密。

3.3 数字签名

数字签名是区块链身份认证的核心机制。

3.3.1 签名过程

1. 创建签名：用私钥对消息 Hash 进行”加密”
2. 验证签名：用公钥”解密”并比对 Hash

```
签名流程：
消息 -> Hash函数 -> Hash值 -> 私钥签名 -> 签名结果
验证流程：
签名结果 + 公钥 -> 解密出Hash值 -> 与消息的新Hash比对
```

重点：

- 签名 = 私钥加密（Hash 值）
- 验证 = 公钥解密（比对 Hash）
- 只有私钥持有者能创建有效签名
- 任何人都能验证签名真伪
- 签名不可伪造、不可抵赖

3.3.2 区块链中的签名应用

- 交易签名：证明交易确实由资产所有者发起
- 区块签名：证明区块确实由矿工/验证者生成
- 智能合约调用：证明调用者身份

3.4 Merkle Tree（默克尔树）

Merkle Tree 是区块链中组织交易数据的核心数据结构。

3.4.1 结构和原理

```
交易列表: [Tx1, Tx2, Tx3, Tx4]

        [Root Hash]           <- Merkle Root（存区块头）
         /        \
   [Hash01]      [Hash23]
    /    \\        /    \
[Tx1]  [Tx2]  [Tx3]  [Tx4]   <- 叶子节点（交易Hash）

计算过程:
Hash0 = Hash(Tx1)
Hash1 = Hash(Tx2)
Hash2 = Hash(Tx3)
Hash3 = Hash(Tx4)
Hash01 = Hash(Hash0 + Hash1)
Hash23 = Hash(Hash2 + Hash3)
Root = Hash(Hash01 + Hash23)
```

3.4.2 Merkle Tree 的优势

1. 快速验证：验证一笔交易是否存在，只需要 $O(\log n)$ 的 Hash 计算
2. 轻节点友好：SPV（简单支付验证）节点只需下载区块头（80 字节），不需要下载所有交易

3. 防篡改：任何一笔交易被修改，Merkle Root 就会改变，立即被发现

```
验证 Tx2 是否在区块中：
路径：Tx2 -> Hash(Tx3) -> Hash0 -> ... -> Merkle Root
如果最终计算得到的Merkle Root与区块头中的一致 -> 交易存在
```

重点：Merkle Root 存储在区块头中。如果交易列表中有任何一笔被修改，其 Merkle Root 就会改变。注意 Merkle Tree 是二叉树，节点数是偶数（奇数时复制最后一个节点）。

4 区块链底层数据结构

4.1 区块结构

一个区块由两部分组成：区块头（**Block Header**）和区块体（**Block Body**）。

4.1.1 区块头（Block Header）

固定 80 字节，包含 6 个字段：

LightBG 字段	长度	说明
Version	4 字节	区块版本号
Previous Block Hash	32 字节	前一个区块的 Hash（链式结构）
Merkle Root	32 字节	所有交易的 Merkle Tree 根 Hash
Timestamp	4 字节	区块创建时间（Unix 时间戳）
Bits	4 字节	难度目标（编码后的目标值）
Nonce	4 字节	随机数（PoW 挖矿用）

重点：区块头总共 80 字节，是 PoW 工作量证明的直接操作对象。父区块哈希将区块串成链。Nonce 是 4 字节无符号整数，意味着在 4 字节范围内（大约 43 亿次）找不到有效 Hash 时，需要其他策略（如改变 Coinbase 交易中的 ExtraNonce）。

4.1.2 区块体（Block Body）

包含交易列表，大小不固定：

- 交易数量（变长整数）
- 交易列表（每笔交易 200-500 字节不等）

  比特币区块大小限制：1MB（隔离见证后 4MB）

4.2 交易结构（UTXO 模型）

比特币使用 **UTXO**（**Unspent Transaction Output**）模型，不是账户余额模型。

4.2.1 什么是 UTXO？

UTXO = 未花费的交易输出。可以理解为" 区块链上的零钱"。

```
类比现实：比特币的"余额"就像你钱包里的现金。
  - 你有：1张100元、2张50元、5张10元
  - 钱包余额 = 100 + 50×2 + 10×5 = 250元

比特币也一样：
  - UTXO #A: 0.5 BTC（来自某笔交易）
  - UTXO #B: 1.2 BTC（来自另一笔交易）
钱包余额 = 1.7 BTC
```

4.2.2 交易过程

```
小明要转1 BTC给小红：

小明的UTXO：
  - UTXO #1: 0.6 BTC
  - UTXO #2: 0.8 BTC
  - 总计：1.4 BTC

交易创建：
  输入：引用 UTXO #1 和 UTXO #2（作为花费）
  输出：
  - 新建 UTXO #A: 1.0 BTC（归属小红）
  - 新建 UTXO #B: 0.399 BTC（归属小明，找零）
  - 手续费：0.001 BTC（给矿工）

结果：
  - UTXO #1 和 #2 被标记为"已花费"
  - UTXO #A 和 #B 成为新的"未花费输出"
```

重点：UTXO 模型的核心是" 输入引用过去的输出，产生新的输出"。理解 UTXO vs 账户余额模型的区别：银行是" 记账"（A 账户减，B 账户加），比特币是" 记录每笔零钱的归属"（UTXO 的流转）。以太坊使用的是账户余额模型。

5 共识算法

共识算法解决的是分布式系统中如何达成一致的问题。

5.1 拜占庭将军问题

由 Leslie Lamport 等人于 1982 年提出，是区块链共识机制最重要的理论来源。

```
情境：10位拜占庭将军分别带领一支军队包围了一座敌人城市。

问题：
- 将军们只能通过信使传递消息（没有电话、没有网络）
- 其中一些将军可能是叛徒，会发送错误信息
- 忠诚的将军们需要就"进攻还是撤退"达成一致

问题：如何让所有忠诚的将军就"进攻还是撤退"达成一致的行动计划？
```

将将军视为区块链节点，信使视为网络消息，叛徒视为恶意节点。

拜占庭容错（**BFT**）：系统能容忍最多 f 个恶意节点，仍能达成一致。

- $n \geq 3f + 1$：系统能容忍最多 f 个恶意节点（即恶意节点不超过总数的 1/3）
- 10 个将军，最多容忍 3 个叛徒
- 100 个节点，最多容忍 33 个恶意节点

5.2 PoW —工作量证明（Proof of Work）

5.2.1 核心思想

谁算力强，谁有话语权。通过消耗计算资源（算力）来竞争记账权。

矿工不断改变一个叫 Nonce 的随机数，对区块头反复计算 Hash，直到 Hash 值小于某个目标值。

```
目标：Hash(区块头 + Nonce) $<$\texttt{0000FFFFFFFFFFFFFFFF}\\ \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFF}

矿工尝试：
Nonce=0     -> Hash=8f8e2a1b3c4d5e6f...
Nonce=1     -> Hash=3a4b5c6d7e8f9a0b...
Nonce=...（数十亿次尝试）
Nonce=12345678 -> Hash=00001a2b3c4d5e6f...  □ 找到啦！
```

5.2.2 PoW 的特点

- 优点：
  - 安全性高：攻击成本极高（需要控制 51% 算力）
  - 去中心化程度高：任何人都可以参与挖矿
  - 公平性：算力即权力，规则透明
- 缺点：

  - 能耗巨大：比特币年耗电量超过阿根廷全国
  - 效率低：每秒只能处理 3-7 笔交易
  - 算力集中：矿池控制大部分算力，有中心化趋势

重点：PoW 的核心是”算力竞争，谁先找到有效 Hash 谁赢”。难度由 Bits 字段编码。51% 攻击是 PoW 的最大威胁，但实际攻击成本远超收益。比特币使用 SHA-256d（两次 SHA-256）作为 PoW 函数。

5.3 PoS —权益证明（Proof of Stake）

5.3.1 核心思想

根据持有的代币数量和时间来选择区块验证者。持有的越多、被锁定的时间越长，被选中出块的概率越高。类似于”股东大会投票”，股份越多，话语权越大。

```
PoS选择公式（简化）：
选择概率 $\propto$ 质押代币数量 × 质押时间

示例：
  - 持有1000 ETH，锁定1年 -> 权重 = 1000 × 1 = 1000
  - 持有500 ETH，锁定2年 -> 权重 = 500 × 2 = 1000
  - 权重相同，随机选一个
```

以太坊 2.0 的 PoS 使用随机化（**Randomized**）方法——RANDAO + VDF（可验证延迟函数）产生随机数，结合质押权重选择出块者。

5.3.2 PoS 的特点

- 优点：
  - 能耗极低：比 PoW 节能 99.95%
  - 效率高：出块速度快，每秒可处理数千笔交易
  - 攻击成本高：攻击者需要购买并质押大量代币，攻击后代币会贬值
- 缺点：
  - 富者愈富：持币多的人更容易获得奖励
  - 无利害关系（Nothing at Stake）：验证者可能在多个分叉上同时签名
  - 初始分配问题：代币初始分配可能不公平

重点：PoS 中”质押”（Stake）意味着将代币作为保证金。如果验证者作恶，其质押的代币会被罚没（Slashing）。以太坊 PoS 最多可罚没验证者 32 ETH。这比 PoW 的 51% 攻击更经济高效，能耗降低约 99.95%。

5.4 DPoS —委托权益证明

5.4.1 核心思想

代币持有者投票选举出少数节点（如 21 个）负责出块和验证。类似于” 代议制民主”，大家投票选代表。

```
EOS的DPoS机制：
1. EOS持有者投票选出21个"区块生产者"（BP）
2. 21个BP轮流出块（每3秒一个区块）
3. BP获得区块奖励，需要回馈社区
4. 如果BP表现不好，持有者可以随时投票罢免
```

5.4.2 DPoS 的特点

- 优点：
  - 效率极高：每秒可处理数千甚至上万笔交易
  - 能耗极低：不需要大规模算力竞争
- 缺点：
  - 中心化程度高：只有 21 个节点控制网络
  - 投票率低：大多数代币持有者不参与投票
  - 去中心化程度降低——节点数量少，更容易被操控

5.5 PBFT —实用拜占庭容错

5.5.1 核心思想

通过多轮投票达成共识，只要恶意节点不超过总数的 1/3，系统就能正常工作。

5.5.2 三阶段协议

1. **Request**：客户端向主节点发送请求
2. **Pre-Prepare**：主节点验证请求，广播给所有节点
3. **Prepare**：节点验证并广播 Prepare 消息
4. **Commit**：节点收到足够多 Prepare 后，广播 Commit 消息
5. **Reply**：节点收到足够多 Commit 后，执行并回复客户端

内部资料

PBFT 的容错能力：

- 总节点数 n，恶意节点数 f
- 系统能正常工作的条件：$n \geq 3f + 1$
- 即：恶意节点不超过总数的 1/3
- 4 个节点可以容忍 1 个恶意节点
- 10 个节点可以容忍 3 个恶意节点

5.5.3 PBFT 的特点

- 优点：
  - 即时确认：交易一旦写入就最终确定，不需要等待多个区块确认
  - 高吞吐量：每秒可处理数千笔交易
  - 确定性：不会出现分叉
- 缺点：
  - 扩展性差：节点数量增加，通信复杂度 $O(n^2)$ 急剧上升
  - 中心化趋势：通常用于联盟链，节点数量有限
  - 不适合公链：公链节点太多，PBFT 通信开销太大

重点：

- PBFT 适合联盟链（节点少、信任度较高）
- PoW/PoS 适合公链（节点多、无需许可）
- PBFT 是" 投票" 机制，PoW 是" 算力竞争" 机制
- 联盟链常用 PBFT（如 Hyperledger Fabric），公链用 PoW/PoS

5.6 Raft —非拜占庭容错共识

5.6.1 核心思想

Raft 是非拜占庭容错算法，假设所有节点都是诚实的，只处理节点故障（宕机、网络分区），不处理恶意节点。

5.6.2 三种角色

- **Leader**（领导者）：负责处理所有客户端请求，复制日志到 Follower
- **Follower**（跟随者）：接收并存储 Leader 的日志，被动响应
- **Candidate**（候选人）：Leader 宕机时，Follower 变成 Candidate，竞选新 Leader

5.6.3 Raft vs PBFT

LightBG 特性	**Raft**	**PBFT**
容错类型	崩溃容错（非恶意）	拜占庭容错（含恶意节点）
节点假设	所有节点诚实	最多 1/3 节点恶意
通信复杂度	$O(n)$	$O(n^2)$
性能	更高（消息少）	较低（多轮投票）
适用场景	私有链、数据库复制	联盟链、跨机构协作
代表项目	etcd、Consul	Hyperledger Fabric

重点：Raft vs PBFT 的区别：

- Raft 假设节点只是故障，不会作恶（崩溃容错）
- PBFT 假设可能有节点作恶（拜占庭容错）
- 私有链/内网用 Raft，联盟链用 PBFT

6 智能合约

6.1 什么是智能合约？

智能合约（Smart Contract）是运行在区块链上的程序代码，可以自动执行合约条款。

智能合约的核心特点：

1. 自动执行：满足条件自动触发，无需人工干预
2. 不可篡改：一旦部署，代码无法修改
3. 透明公开：代码对所有参与者可见
4. 去中心化：在区块链网络的所有节点上运行

6.1.1 生活化类比

Nick Szabo 举了一个经典例子：自动售货机。

```
传统合同：
  1. 你和小明签合同："你给我100元，我给你一个商品"
2. 你给了100元  3. 小明可能反悔不给你商品
  4. 你需要去法院起诉

自动售货机（智能合约）：
  1. 你投入100元（满足条件）
  2. 你按下可乐按钮（发出指令）
  3. 机器自动吐出可乐（合同自动执行）
  ——整个过程不需要人工介入，不需要法院介入！
```

Nick Szabo 认为，法律合同的执行成本很高，而自动化的数字合同可以大幅降低成本。但 1994 年时没有可信的执行环境，直到以太坊出现。

6.2 以太坊和 EVM

6.2.1 以太坊的诞生

Vitalik Buterin（V 神）2013 年提出以太坊概念，2015 年正式上线。

以太坊 = 比特币的区块链 + 图灵完备的虚拟机

- 比特币的脚本系统：简单、非图灵完备（不能循环）
- 以太坊的 EVM：图灵完备，可以执行任意复杂逻辑

6.2.2 EVM（以太坊虚拟机）

EVM 是以太坊智能合约的运行环境：

- 每个以太坊节点都运行 EVM
- 智能合约代码在 EVM 中执行
- EVM 是沙箱环境，合约不能访问外部网络、文件系统等
- 合约之间可以互相调用

6.2.3 Gas 机制

执行智能合约需要消耗 **Gas**，防止无限循环和资源滥用。

```
简单转账：21000 Gas（固定）
执行一次算术运算：3 Gas
存储一个32字节数据：20000 Gas
执行一个合约函数：取决于函数复杂度

Gas计算示例：
  用户A调用一个复杂的DeFi合约：
  - 基本操作Gas：50000
  - 存储写入Gas：80000
  - Gas Price：50 Gwei（0.00000005 ETH）
  - 总费用 = 130000 × 50 Gwei = 0.0065 ETH
```

重点：Gas 是防止无限循环和资源滥用的机制。Gas Limit 是上限，防止意外超额。EVM（以太坊虚拟机）是智能合约的运行环境，与比特币的脚本系统不同。

6.3 Solidity 合约示例

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint256 storedData;

    function set(uint256 x) public {
        storedData = x;
    }

    function get() public view returns (uint256) {
        return storedData;
    }
}
```

调用这个合约：

```
set(42)  ->  交易被广播到网络
           ->  矿工打包到区块
           ->  所有节点执行set函数
           ->  storedData变成42
           ->  状态更新到区块链

get()    ->  本地查询，不消耗Gas
           ->  返回42
```

7 区块链的类型

7.1 公链（Public Blockchain）

- 特点：完全开放，任何人可以参与，无需许可
- 代表：比特币、以太坊
- 共识：PoW、PoS
- 优势：去中心化程度高，抗审查，全球可用
- 劣势：性能低，能耗高，监管困难

7.2 联盟链（Consortium Blockchain）

- 特点：多个机构共同维护，需要许可才能加入
- 代表：Hyperledger Fabric、R3 Corda、FISCO BCOS
- 共识：PBFT、Raft
- 优势：性能高，可监管，适合企业应用
- 劣势：去中心化程度较低

7.3 私链（Private Blockchain）

- 特点：单一机构控制，完全中心化
- 用途：企业内部审计、数据共享
- 优势：效率最高，完全可控
- 劣势：去中心化程度最低

重点：联盟链是中国主推方向，在政务、金融、供应链等领域广泛落地。应重点掌握联盟链的 PBFT/Raft 共识机制和实际应用场景。

8 区块链应用领域

8.1 数字货币与金融应用

8.1.1 比特币（Bitcoin）

比特币是第一个成功的加密货币，开创了区块链时代。

比特币的核心设计：

1. 总量恒定：2100 万枚，通过算法控制发行速度
2. 减半机制：每 4 年产量减半，控制通胀
3. 挖矿奖励：当前区块奖励为 6.25 BTC（2024 年减半后为 3.125 BTC）
4. 价值存储：被称为” 数字黄金”，总量稀缺性支撑价值

8.1.2 稳定币（Stablecoin）

稳定币是为了解决加密货币价格波动问题而设计的代币。

- 法币抵押型：USDT、USDC，1:1 锚定美元
  - 发行方持有等值美元储备
  - 用户可随时兑换回法币
  - 用于加密货币交易对定价
- 加密资产抵押型：DAI，通过超额抵押加密资产发行
  - 抵押率通常 >150%
  - 智能合约自动清算
  - 去中心化发行
- 算法型：通过算法调节供应量维持价格（风险较高）

8.1.3 央行数字货币（CBDC）

- 数字人民币（**e-CNY**）：中国央行发行的法定数字货币
  - 双层运营：央行-> 商业银行-> 公众
  - 可控匿名：小额匿名，大额可追溯
  - 双离线支付：无网络也能完成交易
  - 智能合约：支持条件支付、定向用途
- 与加密货币的区别：
  - CBDC 是法币，有国家信用背书
  - 中心化发行和管理
  - 不是投资品，是支付工具

重点：稳定币是连接传统金融和加密世界的桥梁。CBDC 是法币的数字化形式，与比特币等加密货币有本质区别。

8.2 资产数字化与供应链金融

8.2.1 资产数字化（Asset Tokenization）

将实物资产或权益转化为区块链上的数字代币。

内部资料

资产数字化的优势：

1. 流动性提升：传统非流动性资产（房地产、艺术品）可以碎片化交易
2. 降低门槛：投资者可以购买资产的一部分（如 1/1000 的房产份额）
3. 透明可追溯：资产所有权变更全程记录在链上
4. 降低中介成本：减少律师、公证等中介环节

可数字化的资产类型：

- 实物资产：房地产、黄金、艺术品、收藏品
- 金融资产：股票、债券、基金份额、保险
- 知识产权：专利、版权、商标
- 其他权益：会员权益、积分、门票

8.2.2 供应链金融

供应链金融解决中小企业融资难问题。

```
传统供应链金融的问题：
1. 核心企业确权难：核心企业不愿为上下游中小企业背书
2. 信息不对称：银行难以验证交易真实性
3. 融资成本高：中小企业信用不足，利率高
4. 资金周转慢：应收账款账期长

区块链解决方案：
1. 应收账款上链：核心企业应付账款转化为链上凭证
2. 可拆分流转：供应商可以将应收账款拆分转让给多级供应商
3. 智能合约自动执行：到期自动付款
4. 降低融资成本：银行可以直接验证交易真实性
```

典型案例：

- 蚂蚁链：”双链通”平台，服务超过 1 万家中小企业
- 微众银行：供应链金融平台，基于区块链的应收账款融资
- 京东智臻链：供应链溯源 + 金融一体化平台

重点：资产数字化通过 Token 化提升流动性。供应链金融通过区块链解决信任问题，让核心企业的信用可以传递给多级供应商。

8.3 政务与存证

8.3.1 电子政务

区块链在政务服务中的应用：

- 电子证照
  - 身份证、户口本、结婚证等上链存储
  - 跨部门共享，减少重复提交
  - 防篡改，提高可信度
  - 代表：广东省”粤省事”、浙江省”浙里办”
- 政务数据共享
  - 打破部门数据孤岛
  - 数据使用可追溯，保护隐私
  - 提高政务服务效率
- 公共资源交易
  - 招投标信息上链，防止暗箱操作
  - 确保公平公正

8.3.2 司法存证

区块链存证的优势：

1. 时间戳证明：精确记录证据产生时间
2. 防篡改：证据一旦上链无法修改
3. 司法认可：最高人民法院已明确区块链存证的法律效力
4. 降低成本：无需公证处，降低存证成本

典型案例：

- 北京互联网法院”天平链”：全国首个司法区块链平台
  - 已存证超过 1 亿条数据
  - 支持版权保护、网络侵权等案件
- 杭州互联网法院：司法区块链平台
- 广州互联网法院：”网通法链”平台

重点：区块链存证已被司法认可。最高法规定：当事人提交的区块链存证证据，经核验一致的，法院应当确认其真实性。

8.4 数字身份

- DID（去中心化身份），用户自主控制
- 可验证凭证（VC），保护隐私
- 应用：学历认证、职业资格认证

9 中国区块链发展与国密算法

9.1 国密算法概述

中国自主研发的一系列商用密码算法，包括：

- **SM2**：非对称加密算法（对应 ECDSA/RSA）
- **SM3**：Hash 算法（对应 SHA-256）
- **SM4**：对称加密算法（对应 AES）

9.2 SM2 —非对称加密算法

9.2.1 什么是 SM2?

SM2 是中国自主研发的非对称加密算法，功能类似于国际上的 RSA 和 ECDSA，但采用了不同的数学原理。

SM2 vs 国际算法的对应关系：

- SM2 ECDSA（椭圆曲线数字签名算法）——用于数字签名
- SM2 RSA ——用于加密
- 但 SM2 基于椭圆曲线，比 RSA 更高效、更安全

9.2.2 SM2 的核心概念

1. 密钥对

- 私钥：256 位随机数（64 个十六进制字符），必须保密
- 公钥：从私钥通过椭圆曲线计算得出，可以公开
  - 格式：04 + x 坐标 (32 字节) + y 坐标 (32 字节)
  - 总长度：65 字节（130 个十六进制字符）

2. 椭圆曲线参数 SM2 使用特定的椭圆曲线（sm2p256v1），定义了：

- 曲线方程：$y^2 = x^3 + ax + b$（特定的 a、b 值）
- 基点 G：曲线上的一个固定点
- 阶 n：一个大素数，确保安全性

9.2.3 SM2 的两大功能

功能 1：数字签名

用于证明消息确实来自私钥持有者，不可抵赖。

```
签名过程（简化理解）：
1. 计算消息Hash: e = Hash(message)   （使用SM3算法计算消息摘要）
2. 生成随机数k（每次签名都不同，必须保密）
3. 计算椭圆曲线点: (x1, y1) = k * G   （G是基点）
4. 计算r = (e + x1) mod n
5. 计算s = ((1 + d)^-1 * (k - r*d)) mod n
   其中d是私钥，n是曲线阶
6. 签名结果: (r, s) ——两个大整数

验证过程：
1. 计算消息Hash: e' = Hash(message)
2. 用公钥P和签名(r, s)验证等式是否成立
3. 验证通过 → 签名有效（消息确实来自私钥持有者）
4. 验证失败 → 签名无效或消息被篡改
```

功能 2：加密解密

用于保护数据机密性，只有私钥持有者能解密。

```
加密过程：
1. 生成随机数k
2. 计算椭圆曲线点: C1 = k * G（临时公钥）
3. 计算共享密钥: S = k * 接收方公钥
4. 用共享密钥加密消息 → C2
5. 计算校验值: C3 = SM3(S + message)
6. 密文: (C1, C2, C3)

解密过程：
1. 用私钥计算: S = 私钥 * C1
2. 用共享密钥解密C2得到消息
3. 验证C3是否正确
4. 正确 → 解密成功
```

SM2 考试重点：

- SM2 是基于椭圆曲线的非对称加密算法
- 私钥 256 位，公钥 512 位（04+x+y 格式）
- 签名结果是两个数 (r, s)
- 安全性基于椭圆曲线离散对数难题（ECDLP）
- 相同安全强度下，SM2 比 RSA 快 10 倍以上

9.3 SM3 —Hash 算法

9.3.1 什么是 SM3?

SM3 是中国自主研发的密码学 **Hash** 函数，功能类似于 SHA-256，但算法完全不同。

SM3 的核心特性：

- 输入：任意长度（理论上无限制）
- 输出：固定 256 位（64 个十六进制字符）
- 单向性：无法从 Hash 反推原始数据
- 雪崩效应：输入改变 1bit，输出完全不同
- 抗碰撞：极难找到两个不同输入产生相同 Hash

9.3.2 SM3 vs SHA-256

特性	**SM3**	**SHA-256**
输出长度	256 位	256 位
输出格式	64 个十六进制字符	64 个十六进制字符
内部结构	16 轮压缩函数	64 轮压缩函数
初始值	特定常数	自然常数的小数部分
安全性	国密标准	国际通用
相同输入结果	不同	不同

9.3.3 SM3 的计算过程（简化）

```
SM3计算步骤：

1. 消息填充：
   - 在消息后添加1个1和若干个0
   - 最后添加64位的消息长度
   - 确保总长度是512的倍数

2. 消息扩展：
   - 将消息分成512位的块
   - 每个块扩展成132个字（W0-W67, W'0-W'63）

3. 迭代压缩（16轮）：
   - 初始化8个寄存器（A-H）
   - 每轮更新寄存器的值
   - 使用布尔函数、移位运算、模加

4. 输出结果：
   - 将最终的8个寄存器值拼接
   - 得到256位的Hash值
```

实际应用示例：

```
Python使用SM3：
from gmssl import sm3

message = b"Hello SM3"
hash_value = sm3.sm3_hash(message)
print(f"SM3 Hash: {hash_value}")
# 输出：66c7f0f462eeedd9d1f2d46b5e9d0b3e8e8e8e8e8e8e8e8e8e8e8e8e8e8e8e8

对比SHA-256：
import hashlib
sha256_value = hashlib.sha256(message).hexdigest()
print(f"SHA-256: {sha256_value}")
# 输出：185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

结论：相同输入，SM3和SHA-256输出完全不同！
```

SM3 考试重点：

- SM3 输出 256 位（64 个十六进制字符）
- SM3 和 SHA-256 输出长度相同，但结果不同
- SM3 用于：数据完整性验证、数字签名中的 Hash 计算、区块链中的交易 Hash
- SM3 是国密标准，中国政府和企业优先使用

9.4 SM4 —对称加密算法

9.4.1 什么是 SM4?

SM4 是中国自主研发的对称加密算法，功能类似于 AES（高级加密标准），但算法设计完全不同。

对称加密的核心特点：

- 加密和解密使用同一个密钥
- 密钥必须安全地分发给通信双方
- 加密速度快，适合大数据量加密
- SM4 密钥长度：128 位（16 字节）
- SM4 分组长度：128 位（16 字节）

9.4.2 SM4 vs AES 对比

特性	**SM4**	**AES**
密钥长度	128 位（固定）	128/192/256 位
分组长度	128 位	128 位
加密轮数	32 轮	10/12/14 轮
结构	非平衡 Feistel 网络	替换-置换网络（SPN）
S 盒	固定 8 位 S 盒	可逆 8 位 S 盒

9.4.3 SM4 的加密过程

```
SM4加密步骤（简化）：

1. 密钥扩展：
   - 输入：128位密钥
   - 输出：32个轮密钥（每轮一个）
   - 使用FK常数和CK常数进行扩展

2. 轮函数（32轮）：
   每轮操作：
   - X = 当前状态（4个字）
   - 非线性变换τ：使用S盒进行字节替换
   - 线性变换L：循环移位和异或
   - 轮密钥加：与轮密钥异或
   - 更新状态

3. 反序变换：
   - 32轮结束后，将结果反序
   - 得到128位密文
```

9.4.4 SM4 的工作模式

```
ECB模式（电子密码本）——不推荐：
┌──────────┐  ┌──────────┐  ┌──────────┐
│ 明文块1 │  │ 明文块2 │  │ 明文块3 │
```

```
└──────┬──────┘   └──────┬──────┘   └──────┬──────┘
       │                 │                 │
   ┌───┴───┐         ┌───┴───┐         ┌───┴───┐
   │ SM4   │         │ SM4   │         │ SM4   │   ← 相同密钥，独立加密
   │加密   │         │加密   │         │加密   │
   └───┬───┘         └───┬───┘         └───┬───┘
       │                 │                 │
   ┌───┴───┐         ┌───┴───┐         ┌───┴───┐
   │密文块1│         │密文块2│         │密文块3│
   └───────┘         └───────┘         └───────┘

缺点：相同明文块产生相同密文块，不安全

CBC模式（密码分组链接）——推荐：
┌───────┐         ┌───────┐         ┌───────┐
│明文块1│         │明文块2│         │明文块3│
└───┬───┘         └───┬───┘         └───┬───┘
    │                 │                 │
    │  IV(初始向量)   │                 │
    │   │             │   │             │
    └───┼─────────────┼───┼─────────────┘
        ▼             │   ▼             │
      ┌───┐           │ ┌───┐           │
      │XOR│           │ │XOR│           │
      └───┘           │ └───┘           │
        │             │   │             │
      ┌───┐           │ ┌───┐           │
      │SM4│           │ │SM4│           │
      │加密│           │ │加密│           │
      └───┘           │ └───┘           │
        │             │   │             │
    ┌───┴───┐         │ ┌───┴───┐         │
    │密文块1│         │ │密文块2│         │
    └───────┘         │ └───────┘         │

优点：相同明文产生不同密文，更安全
需要：随机IV（初始向量），每次加密都不同
```

9.4.5 SM4 的实际应用

```
Python使用SM4（使用gmssl库）：

from gmssl.sm4 import CryptSM4, SM4_ENCRYPT, SM4_DECRYPT

# 密钥（16字节）
key = b'0123456789abcdef'

# 初始向量（16字节，CBC模式需要）
iv = b'abcdef0123456789'

```

```
# 创建SM4对象
sm4_crypt = CryptSM4()
# 加密sm4_crypt.set_key(key, SM4_ENCRYPT)
plaintext = b'Hello, SM4!'
# 填充到16字节倍数
padding_len = 16 - len(plaintext) % 16
plaintext += bytes([padding_len] * padding_len)
ciphertext = sm4_crypt.crypt_cbc(iv, plaintext)
print(f"密文: {ciphertext.hex()}")

# 解密
sm4_crypt.set_key(key, SM4_DECRYPT)
decrypted = sm4_crypt.crypt_cbc(iv, ciphertext)
# 去除填充
padding_len = decrypted[-1]
decrypted = decrypted[:-padding_len]
print(f"解密: {decrypted.decode()}")
```

SM4 考试重点：

- SM4 是对称加密算法，加密解密用同一个密钥
- 密钥长度 128 位，分组长度 128 位，32 轮加密
- 推荐使用 CBC 模式（需要 IV），不推荐 ECB 模式
- SM4 适合加密大量数据（比非对称加密快 100 倍以上）
- 应用场景：数据传输加密、文件加密、数据库加密

9.5 国密算法总结

算法	类型	对应国际算法	主要用途
SM2	非对称加密	ECDSA/RSA	数字签名、密钥交换
SM3	Hash 函数	SHA-256	数据完整性、签名 Hash
SM4	对称加密	AES	大数据量加密

国密算法的优势：

1. 自主可控：不依赖国外技术，保障国家安全
2. 性能优化：针对中国硬件环境优化
3. 合规要求：中国政府和企业项目必须使用
4. 安全性：经过严格密码学分析和测试
5. 生态完善：支持硬件加速（加密卡、加密芯片）

10 数据结构基础

10.1 链表（Linked List）

```
普通链表：
  节点A -> 节点B -> 节点C -> 节点D
  指针存储下一个节点的内存地址

Hash指针链表（区块链）：
  节点A -> 节点B -> 节点C -> 节点D
  节点A的"指针" = Hash(节点B的内容)

  优势：即使有人修改了节点C的数据，
  节点D中存储的Hash(节点C的内容) 立刻失效，
  整个篡改行为被发现。
```

重点：区块链使用链表结构，每个区块通过 Hash 指针指向前一个区块。Hash 指针 = 数据内容 Hash + 位置指针，即使数据移动，Hash 值仍然有效，保证链的完整性。

10.2 树（Tree）

10.2.1 二叉树

每个节点最多有两个子节点。

10.2.2 Merkle Tree

见第 3 章，区块链核心数据结构。

10.2.3 MPT（Merkle Patricia Tree）

以太坊使用的数据结构，结合了：

- Merkle Tree：验证数据完整性
- Patricia Trie：高效存储和检索

重点：以太坊使用 MPT 而非比特币的简单 Merkle Tree，因为 MPT 支持增删改查操作（而 Merkle Tree 主要用于验证完整性）。MPT 是理解以太坊区块结构的关键。

11 链上身份（DID）

11.1 什么是 DID？

DID（Decentralized Identifier，去中心化标识符）是用户自主控制的数字身份，不依赖中心化机构（如政府、公司）。

11.1.1 DID 的格式

```
did:example:123456789abcdefghi
├── did: 方案前缀
├── example: DID方法名（指定操作DID的区块链/网络）
└── 123456789abcdefghi: 唯一标识符（由DID方法生成）
```

11.1.2 DID 的生成

用户通过加密算法生成 DID 和密钥对，DID 本身不包含任何个人身份信息（隐私设计）。

```
DID密钥生成流程：
1. 生成ECDSA secp256k1密钥对（或SM2）
   私钥: 0x7c9e6a5d...
   公钥: 0x04a8e7b6...
2. 计算公钥Hash: SHA256(公钥) -> RIPEMD160 -> Base58
3. 加上方法前缀: did:ethr:0x1234...
4. DID文档存储在以太坊或IPFS上
```

11.1.3 DID 文档（DID Document）

DID 文档描述了与 DID 关联的公钥、服务端点等信息，存储在区块链或去中心化存储上。

```
DID Document示例：
{
  "@context": "https://www.w3.org/ns/did/v1",
  "id": "did:ethr:0x1234abcd...",
  "verificationMethod": [{
    "id": "did:ethr:0x1234#keys-1",
    "type": "EcdsaSecp256k1",
    "controller": "did:ethr:0x1234...",
    "publicKeyHex": "02a8e7b6..."
```

```
  }],
"authentication": ["did:ethr:0x1234...#keys-1"],  "service": [{
    "type": "LinkedDomains",
    "serviceEndpoint": "https://..."
  }]
}
```

DID 文档可以更新的操作（更改公钥、服务端点），但 DID 标识符本身保持不变。

11.2 可验证凭证（VC）

VC（Verifiable Credential，可验证凭证）是由权威机构签发的、关于某个主体的数字化声明。

```
VC示例：学历证书
{
  "@context": ["https://www.w3.org/2018/credentials/v1"],
  "type": ["VerifiableCredential", "UniversityDegreeCredential"],
  "issuer": "did:ethr:0xuniversity...",
  "issuanceDate": "2024-06-01",
  "credentialSubject": {
    "id": "did:ethr:0xstudent...",
    "degree": {
      "type": "BachelorDegree",
      "name": "计算机科学",
      "university": "某某大学"
    }
  },
  "proof": {
    "type": "EcdsaSecp256k1Signature2019",
    "created": "2024-06-01",
    "proofValue": "..."
  }
}
```

VC 的核心特点：

1. 可验证：任何人都可以验证签发者身份和签名
2. 防篡改：任何修改都会导致签名失效
3. 隐私保护：可以选择性披露（只证明”我有本科学历”，不透露具体学校）
4. 自主控制：用户自己存储和管理凭证，不需要中心数据库

重点：DID = 去中心化标识符，由用户自己控制，不依赖中心化机构。三个核心组件：DID 标识符、DID 文档（存公钥）、可验证凭证（VC）。DID 文档存储在区块链或 IPFS 上。

12 实操技能备考要点

12.1 实操技能要点

本章节主要涵盖实操技能相关知识点：

1. 数据结构的基本原理及其在区块链中的应用
2. 主流密码学算法在区块链身份标识方面的应用

12.2 Python 编程要点

12.2.1 Hash 计算

```python
import hashlib

# SHA-256计算
def sha256_hash(data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

# 使用示例
result = sha256_hash("Hello Blockchain")
print(f"SHA-256: {result}")
# 输出: SHA-256: 7
    f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069
```

12.2.2 Merkle Root 计算

```python
import hashlib

def sha256(data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

def merkle_root(txs):
    """计算Merkle Root"""
    if not txs:
        return sha256("")
```

```python

    # 计算所有交易的Hash
hashes = [sha256(tx) for tx in txs]
    # 逐层计算，直到只剩一个Hash
    while len(hashes) > 1:
        # 如果奇数个，复制最后一个
        if len(hashes) % 2 == 1:
            hashes.append(hashes[-1])

        # 两两配对计算父Hash
        parent_hashes = []
        for i in range(0, len(hashes), 2):
            parent = sha256(hashes[i] + hashes[i+1])
            parent_hashes.append(parent)

        hashes = parent_hashes

    return hashes[0]

# 测试
txs = ["tx1", "tx2", "tx3", "tx4"]
print(f"Merkle Root: {merkle_root(txs)}")
```

12.2.3 简单区块链实现

```python
import hashlib
import time

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.mine_block()

    def calculate_hash(self):
        value = str(self.index) + str(self.timestamp) + \
                str(self.data) + str(self.previous_hash) + \
                str(self.nonce)
        return hashlib.sha256(value.encode()).hexdigest()

    def mine_block(self, difficulty=4):
        """PoW挖矿"""
        target = "0" * difficulty
        while True:
            self.hash = self.calculate_hash()
            if self.hash.startswith(target):
```

```
                return self.hash
            self.nonce += 1
class Blockchain:    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, data):
        previous_hash = self.get_latest_block().hash
        new_block = Block(len(self.chain), data, previous_hash)
        self.chain.append(new_block)

    def is_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i-1]

            if current.hash != current.calculate_hash():
                return False
            if current.previous_hash != previous.hash:
                return False
        return True

# 使用示例
blockchain = Blockchain()
blockchain.add_block("First Transaction")
blockchain.add_block("Second Transaction")

print(f"区块链有效: {blockchain.is_valid()}")
for block in blockchain.chain:
    print(f"区块{block.index}: {block.hash[:20]}...")
```

12.3 区块链浏览器实操

12.3.1 比特币浏览器使用

以 blockchain.com 为例，学习如何查询交易信息。

```
实操步骤:
1. 打开 blockchain.com/explorer
2. 在搜索框输入交易ID（TxID）
   示例:
      4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b

   （这是比特币创世区块的第一笔交易）

```

```
3. 查看交易详情:
- 交易哈希（Transaction Hash）    - 区块高度（Block Height）
   - 确认数（Confirmations）
   - 发送方/接收方地址
   - 交易金额
   - 交易费用
   - 时间戳

4. 查看区块信息:
   - 区块哈希
   - 前一区块哈希（形成链式结构）
   - Merkle Root
   - Nonce
   - 难度目标
   - 包含的交易数量
```

12.3.2 以太坊浏览器使用

以 etherscan.io 为例。

```
实操步骤:
1. 打开 etherscan.io
2. 查询账户地址:
   示例: 0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb

3. 查看账户信息:
   - ETH余额
   - 交易数量
   - 持有的代币（Token）
   - 交易历史

4. 查询智能合约:
   - 合约代码（Verified Source Code）
   - 合约ABI
   - 合约创建交易
   - 合约调用记录

5. 查看Gas费用:
   - Gas Price（Gwei）
   - Gas Limit
   - 实际消耗的Gas
   - 交易总费用 = Gas Used × Gas Price
```

重点：区块链浏览器是查看链上数据的窗口。比特币浏览器主要看 UTXO 和区块信息，以太坊浏览器主要看账户、智能合约和 Gas 费用。

12.4 数字签名实操

12.4.1 ECDSA 签名与验证（Python 实现）

```python
import hashlib
import ecdsa
from ecdsa import SigningKey, VerifyingKey, SECP256k1

def generate_keypair():
    """生成密钥对"""
    # 生成私钥
    private_key = SigningKey.generate(curve=SECP256k1)
    # 从私钥生成公钥
    public_key = private_key.get_verifying_key()
    return private_key, public_key

def sign_message(private_key, message):
    """使用私钥签名消息"""
    # 计算消息Hash
    message_hash = hashlib.sha256(message.encode()).digest()
    # 签名
    signature = private_key.sign(message_hash, hashfunc=hashlib.
        sha256)
    return signature

def verify_signature(public_key, message, signature):
    """使用公钥验证签名"""
    try:
        message_hash = hashlib.sha256(message.encode()).digest()
        # 验证签名
        public_key.verify(signature, message_hash, hashfunc=
            hashlib.sha256)
        return True
    except ecdsa.BadSignatureError:
        return False

# 实操示例
if __name__ == "__main__":
    # 1. 生成密钥对
    private_key, public_key = generate_keypair()
    print(f"私钥: {private_key.to_string().hex()}")
    print(f"公钥: {public_key.to_string().hex()}")

    # 2. 要签名的消息
    message = "转账给Alice 1个比特币"
    print(f"\n原始消息: {message}")

    # 3. 签名
    signature = sign_message(private_key, message)
    print(f"签名: {signature.hex()}")

```

```
    # 4. 验证签名
is_valid = verify_signature(public_key, message, signature)
    print(f"\n签名验证结果: {is_valid}")

    # 5. 篡改消息后再验证
    tampered_message = "转账给Bob 100个比特币"
    is_valid_tampered = verify_signature(public_key,
        tampered_message, signature)
    print(f"篡改后验证结果: {is_valid_tampered}")
```

12.4.2 SM2 签名与验证（国密算法）

```
from gmssl import sm2, func

def sm2_sign_verify():
    """SM2签名与验证示例"""
    # 密钥对（十六进制表示）
    private_key = '59
        e2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e2
        '
    public_key = '04' + '8
        d1d8c5df8e4286e0c8d25b602b5503e0ad4c8b6c374b8d9b8f2e8e5e5e5e5e
        ' + \
                  '9
                     f2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42
                     '

    # 创建SM2对象
    sm2_crypt = sm2.CryptSM2(public_key=public_key, private_key=
        private_key)

    # 要签名的消息
    message = b"转账给Alice 100元"

    # 签名
    signature = sm2_crypt.sign(message, private_key)
    print(f"SM2签名: {signature.hex()}")

    # 验证签名
    result = sm2_crypt.verify(signature, message)
    print(f"SM2签名验证结果: {result}")

    return result

# 注意: 实际使用时需要安装gmssl库
# pip install gmssl
```

签名验证的核心逻辑：

1. 签名过程：私钥 + 消息 Hash → 签名值 (r, s)
2. 验证过程：公钥 + 消息 Hash + 签名值 → 验证通过/失败
3. 安全性：没有私钥无法生成有效签名，但任何人都可以用公钥验证
4. 不可抵赖：只有私钥持有者能生成签名，签名者无法否认

12.5 备考建议

1. 理解原理：不要死记硬背，理解每个算法的原理和应用场景
2. 动手实践：至少实现一遍 SHA-256、Merkle Tree、简单区块链
3. 熟悉 **Python**：掌握基本语法、hashlib 库的使用
4. 编码规范：代码缩进、命名、注释
5. 时间分配：理论 45 分钟，实操 60 分钟，合理分配时间

13 附录 A：核心术语速查表

LightBG 术语	解释
区块链	分布式账本技术，数据不可篡改
Hash	单向函数，任意输入产生固定长度输出
SHA-256	安全 Hash 算法，输出 256 位
Merkle Tree	二叉树结构，用于快速验证数据完整性
UTXO	未花费交易输出，比特币的记账模型
PoW	工作量证明，通过算力竞争记账权
PoS	权益证明，通过持币量和时间选择验证者
PBFT	实用拜占庭容错，联盟链常用共识
智能合约	运行在区块链上的自动执行程序
DID	去中心化身份标识符
VC	可验证凭证
SM2/SM3/SM4	中国国密算法
Gas	以太坊执行操作的费用单位

内部资料

14 附录 B：知识体系思维导图

重点速查（按模块整理）

【必须记忆】

- Hash 特性：单向、雪崩、抗碰撞
- SHA-256 输出：64 个十六进制字符 = 256 位
- ECC vs RSA：256 位 ECC ≈ 3072 位 RSA
- 私钥签名、公钥验证
- 区块头 6 个字段：Version、PrevHash、Merkle Root、Timestamp、Bits、Nonce
- UTXO 模型：输入引用过去的输出，产生新的输出
- PoW：算力竞争，找到 Nonce 使 Hash < 目标值
- PoS：质押代币，随机选择验证者
- PBFT：三阶段投票，容忍 < 1/3 恶意节点
- 联盟链 vs 公链：PBFT/Raft vs PoW/PoS
- SM2/SM3/SM4：国密非对称加密、Hash、对称加密
- DID 三要素：标识符、文档、可验证凭证

【理解即可】

- 拜占庭将军问题：叛徒不超过 1/3 时忠诚将军可达成一致
- PoS 根据持币量和时间选择记账者，不消耗算力
- Merkle Tree：根 Hash 代表所有交易，任意一笔修改导致根变化
- 联盟链用 PBFT/Raft，公链用 PoW/PoS
- DID 三要素：DID 标识符 + DID 文档 + 可验证凭证（VC）

【实操相关】

- SHA-256 计算：`hashlib.sha256(data).hexdigest()`
- 区块链结构：区块头（Hash 指针链接）+ 区块体（交易列表）
- Merkle Root 计算：逐层两两 Hash，直到只剩一个根 Hash

15 附录 C：推荐学习资源

- 比特币白皮书：《Bitcoin: A Peer-to-Peer Electronic Cash System》（中本聪）
- 以太坊白皮书：《A Next-Generation Smart Contract and Decentralized Application Platform》（Vitalik Buterin）
- 国密标准：GM/T 0003-2012（SM2）、GM/T 0004-2012（SM3）、GM/T 0002-2012（SM4）
- 在线工具：`blockchain.com/explorer`（比特币浏览器）、`etherscan.io`（以太坊浏览器）
- **Python** 库：`hashlib`（标准库）、`gmssl`（国密算法）

16 附录 D：实操环境搭建指南

16.1 Python 环境准备

1. 安装 **Python 3.8+**：从官网下载安装 `python.org`
2. 安装必要库：

```
pip install hashlib gmssl pycryptodome
```

3. 验证安装：

```
import hashlib
print(hashlib.sha256(b"test").hexdigest())
```

16.2 区块链浏览器使用

1. 比特币浏览器：`blockchain.com/explorer`
   - 输入交易 ID 查询交易详情
   - 查看区块高度、时间戳、交易数量
2. 以太坊浏览器：`etherscan.io`
   - 查询账户余额和交易历史
   - 查看智能合约代码

17 附录 E：模拟试题

17.1 理论知识题

一、选择题（每题 **5** 分）

1. SHA-256 算法的输出长度是 ()

   A 128 位
   B 256 位
   C 512 位
   D 1024 位

2. 比特币使用的共识算法是 ()

   A PoS
   B PBFT
   C PoW
   D Raft

3. PBFT 算法能容忍的恶意节点比例最多为 ()

   A 1/2
   B 1/3
   C 2/3
   D 1/4

4. 以下哪个是中国国密非对称加密算法 ()

   A RSA
   B ECDSA
   C SM2
   D AES

二、简答题（每题 **10** 分）

1. 简述区块链的不可篡改性是如何通过密码学实现的。
2. 解释 UTXO 模型与账户余额模型的区别。
3. 说明 Merkle Tree 在区块链中的作用。

17.2 实操技能题

三、编程题（每题 **20** 分）

1. 编写 Python 函数计算给定字符串的 SHA-256 哈希值，并验证雪崩效应。
   参考答案：

```python
import hashlib

def sha256_hash(data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

# 验证雪崩效应
h1 = sha256_hash("blockchain")
h2 = sha256_hash("blockchaiN")
print(f"原始: {h1}")print(f"修改: {h2}")
print(f"不同位数: {sum(c1 != c2 for c1, c2 in zip(h1, h2))}")
```

2. 实现简单的 Merkle Root 计算函数。

   参考答案：

```python
import hashlib

def sha256(data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

def merkle_root(txs):
    hashes = [sha256(tx) for tx in txs]
    while len(hashes) > 1:
        if len(hashes) % 2 == 1:
            hashes.append(hashes[-1])
        parent_hashes = []
        for i in range(0, len(hashes), 2):
            parent = sha256(hashes[i] + hashes[i+1])
            parent_hashes.append(parent)
        hashes = parent_hashes
    return hashes[0]

# 测试
txs = ["tx1", "tx2", "tx3", "tx4"]
print(f"Merkle Root: {merkle_root(txs)}")
```

17.3 答案速查

题号	答案	题号	答案
1	B	3	B
2	C	4	C